LAPORAN PRAKTIKUM

PRAKTIKUM PENAMBANGAN DATA

PERTEMUAN 1

NUMPY DAN PANDAS



Disusun oleh:

Nama	:	Hanan Fijananto
NIM	:	24/538946/SV/24555
Kelas	:	A2
Dosen Pengampu	:	Dr.Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

A Tujuan

1. Mahasiswa mampu memahami konsep dasar Numpy dan penggunaan array n-dimensi serta berbagai operasi manipulasi array
2. Mahasiswa mampu memahami konsep dasar Pandas dan penggunaan DataFrame serta berbagai operasi manipulasi DataFrame
3. Mahasiswa dapat menerapkan Numpy dan pandas untuk mengolah dan menganalisis data

B Dasar Teori

B.1 Python

Python merupakan bahasa pemrograman komputer yang biasa dipakai untuk membangun situs, software/aplikasi, mengotomatiskan tugas dan melakukan analisis data. Bahasa pemrograman ini termasuk bahasa tujuan umum. Artinya, ia bisa digunakan untuk membuat berbagai program berbeda, bukan khusus untuk masalah tertentu saja.

Berdasarkan survei pengembang Stack Overflow tahun 2022, Python menjadi bahasa pemrograman terpopuler keempat. Sebanyak hampir 50% dari responden mengatakan bahwa mereka menggunakan hampir setengah dari waktu kerja mereka dengan menggunakan bahasa pemrograman ini [1].

B.2 Numpy

NumPy (Numerical Python) adalah library di Python yang dirancang khusus untuk urusan perhitungan ilmiah. Numpy berisi seperti toolbox yang berisi alat-alat canggih untuk mengolah data, matriks, statistik, hingga aljabar linear [2].

B.3 Pandas

Pandas adalah library Python open-source yang menyediakan struktur data (DataFrame) dan alat analisis data yang berkinerja tinggi dan mudah digunakan [3]. Libraries Pandas memiliki format data yang disebut DataFrame, jadi Pandas DataFrame adalah struktur data dua dimensi seperti tabel yang berisi baris dan kolom. Pandas DataFrame berfungsi untuk menyimpan data dalam format grid yang bisa diubah-ubah dengan fleksibilitas yang sangat besar.

Gambaran sederhananya, Pandas DataFrame mirip dengan tabel yang ada di Microsoft Excel. Setiap baris dan kolom akan memiliki label yang bisa digunakan untuk mengakses dan memanipulasi data [4].

C Hasil dan Pembahasan

C.1 Numpy

C.1.1 Install Numpy

Pada praktikum ini, praktikan menggunakan jupyter notebook yang ada di visual studio code. Buat notebook baru dan jalankan perintah berikut untuk menginstall library Numpy.

```
1 !pip install numpy
```

C.1.2 Impor pustaka Numpy

Import pustaka Numpy diperlukan untuk menggunakan fungsi-fungsi yang ada di dalam library Numpy. Alias np digunakan untuk mempersingkat penulisan kode. Jika dijalankan, maka akan muncul output berupa versi Numpy yang digunakan. Berikut adalah contoh kode untuk mengimpor pustaka Numpy.

```
import numpy as np

print(np.__version__)
```

```
2.4.3
```

Gambar 1: Impor pustaka Numpy

C.1.3 Membuat array 1 dimensi

Untuk membuat array 1 dimensi, dapat menggunakan fungsi `np.array()` dengan memasukkan list sebagai argumen. Gambar 2 adalah contoh kode untuk membuat array 1 dimensi.

```
arr = np.array([1, 2, 3, 4, 5])
print("Array: ", arr)

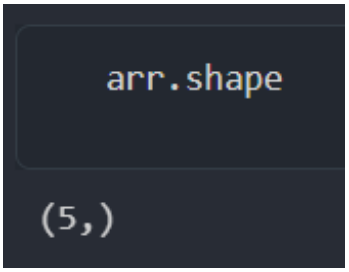
print(f"Cek tipe data: {type(arr)}")
```

```
Array:  [1 2 3 4 5]
Cek tipe data: <class 'numpy.ndarray'>
```

Gambar 2: Membuat array 1 dimensi

C.1.4 Melihat shape atau bentuk array

Untuk melihat bentuk array atau shape array, dapat menggunakan atribut `.shape`. Gambar 3 adalah contoh kode untuk melihat shape array.

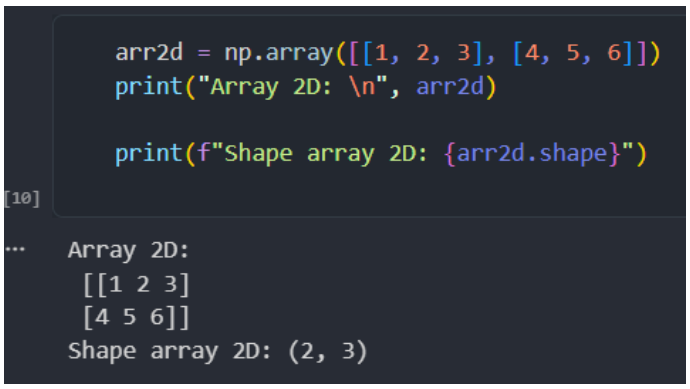


Gambar 3: Melihat shape array

Dapat dilihat pada gambar 3, shape array adalah (5,). Artinya, array tersebut memiliki 5 elemen dan hanya memiliki 1 dimensi.

C.1.5 Membuat array 2 dimensi

Membuat array 2 dimensi dapat dilakukan dengan fungsi yang sama. Namun, argumen yang dimasukkan adalah list yang berisi list-list lain (Nested List). Gambar 4 adalah contoh kode untuk membuat array 2 dimensi.



Gambar 4: Membuat array 2 dimensi

Berdasarkan gambar 4, shape array 2 dimensi yaitu (2, 3). Artinya angka pertama (2) menunjukkan jumlah baris, sedangkan angka kedua (3) menunjukkan jumlah kolom.

C.1.6 Membuat array 3 dimensi

Membuat array 3 dimensi dapat dilakukan dengan fungsi yang sama. Namun, argumen yang dimasukkan adalah list yang berisi list-list lain yang berisi list-list lain (Nested List). Gambar 5 adalah contoh kode untuk membuat array 3 dimensi.

```
arr3d = np.array(
    [
        [
            [1,2,3,4],
            [5,6,7,8]
        ],
        [
            [9,10,11,12],
            [13,14,15,16]
        ],
        [
            [17,18,19,20],
            [21,22,23,24]
        ]
    ]
)

print(arr3d)

print(f"Shape array 3D: {arr3d.shape}")
```

```
[14]
...  [[[ 1  2  3  4]
      [ 5  6  7  8]]

     [[ 9 10 11 12]
      [13 14 15 16]]

     [[17 18 19 20]
      [21 22 23 24]]]
    Shape array 3D: (3, 2, 4)
```

Gambar 5: Membuat array 3 dimensi

C.1.7 Membuat array dengan string sebagai member

Array juga dapat dibuat dengan string sebagai member. Gambar 6 adalah contoh kode untuk membuat array dengan string sebagai member.

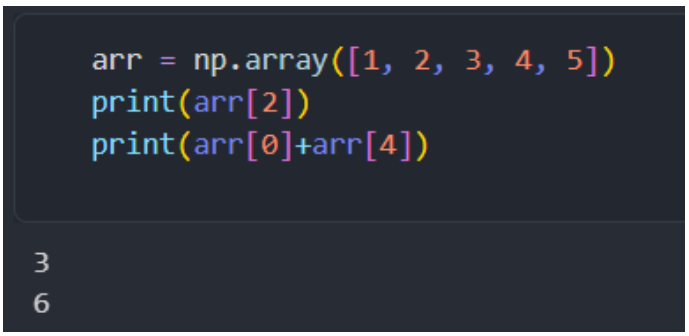
```
arr_str = np.array(["hello", "nama", "saya"])
print(arr_str)
```

```
['hello' 'nama' 'saya']
```

Gambar 6: Membuat array dengan string sebagai member

C.1.8 Melakukan indexing di array 1 dimensi

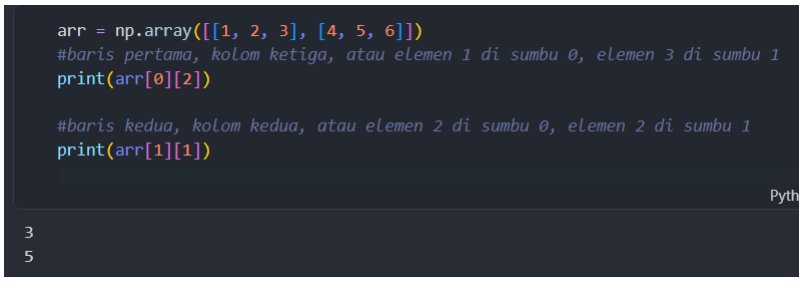
Indexing adalah proses untuk mengakses elemen tertentu dari array. Untuk melakukan indexing di array 1 dimensi, dapat menggunakan tanda kurung siku `[]` dengan memasukkan indeks elemen yang ingin diakses. Gambar 7 adalah contoh kode untuk melakukan indexing di array 1 dimensi.



Gambar 7: Melakukan indexing di array 1 dimensi

C.1.9 Melakukan indexing di array 2 dimensi

Indexing di array 2 dimensi dilakukan dengan menggunakan dua indeks, satu untuk baris dan satu untuk kolom. Gambar 8 adalah contoh kode untuk melakukan indexing di array 2 dimensi.



Gambar 8: Melakukan indexing di array 2 dimensi

C.1.10 Melakukan indexing di array 3 dimensi

Indexing di array 3 dimensi dilakukan dengan menggunakan tiga indeks, satu untuk baris, satu untuk kolom, dan satu untuk kedalaman. Gambar 9 adalah contoh

kode untuk melakukan indexing di array 3 dimensi.

```
arr = np.array([
    [[1, 2, 3], [4, 5, 6]],
    [[10, 11, 12], [13, 14, 14]],
])

#menampilkan angka 11
#sumbu 0 : index 1
#sumbu 1 : index 0
#sumbu 2 : index 1
print(arr[1][0][1])

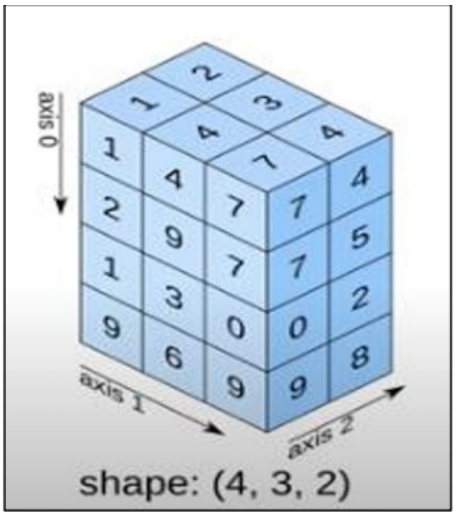
#menampilkan angka 14
#sumbu 0 : index 1
#sumbu 1 : index 1
#sumbu 2 : index 2
print(arr[1][1][2])
```

```
11
14
```

Gambar 9: Melakukan indexing di array 3 dimensi

C.1.11 Membuat array dari contoh

Diberikan array dengan bentuk seperti pada gambar 10.



Gambar 10: Membuat array dari contoh

Kode yang digunakan untuk membuat array tersebut adalah sebagai berikut.

```
    implementasi_array_3d = np.array([
        [[1,2], [4,3],[7,4]],
        [[2,0], [9,0], [7,5]],
        [[1,0], [3,0], [0,2]],
        [[9,0], [6,0], [9,8]]
    ])

    print(implementasi_array_3d)

    print(f"Shape array 3D: {implementasi_array_3d.shape}")
```

```
[[[1 2]
  [4 3]
  [7 4]]

 [[2 0]
  [9 0]
  [7 5]]

 [[1 0]
  [3 0]
  [0 2]]

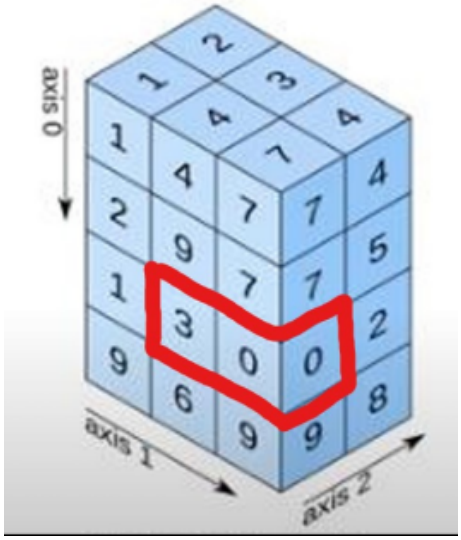
 [[9 0]
  [6 0]
  [9 8]]]
Shape array 3D: (4, 3, 2)
```

Gambar 11: Kode untuk membuat array dari contoh

Berdasarkan gambar 11, array tersebut dibuat dengan menggunakan fungsi `np.array()` dengan memasukkan list yang berisi list-list lain sebagai argumen. Sumbu 0 (axis 0) berisi 4 elemen, sumbu 1 berisi 3 elemen, dan sumbu 2 berisi 2 elemen. Sehingga shape array tersebut adalah (4, 3, 2).

C.1.12 Array slicing

Numpy menerapkan cara yang sama dengan Python untuk melakukan slicing pada array. Slicing adalah proses untuk mengambil sebagian elemen dari array. Gambar 12 adalah contoh gambar yang diberi tanda merah yang akan ditentukan indeks lokasinya seperti pada gambar 13.



Gambar 12: Contoh target slicing

Berdasarkan gambar 12, target slicing berada di sumbu 0 indeks 2, sumbu 1 indeks 1 sampai 2, dan sumbu 2 indeks 0. Sehingga kode untuk melakukan slicing adalah sebagai berikut.

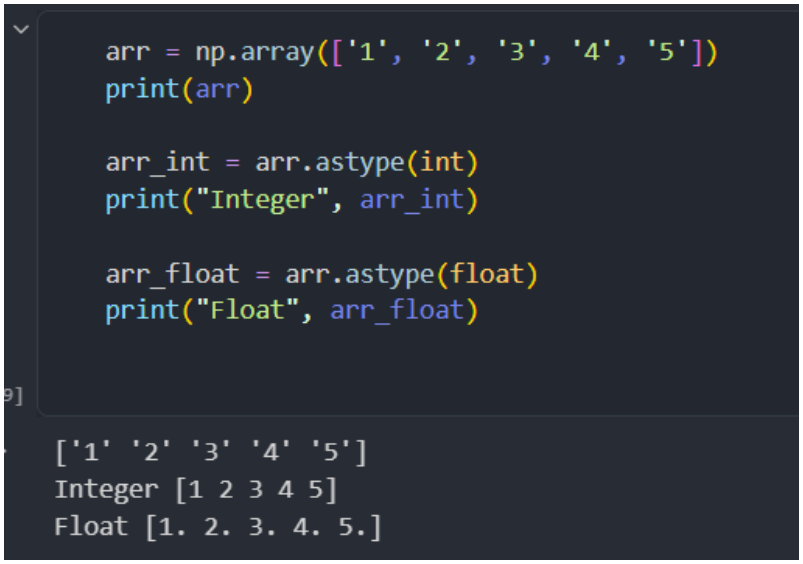
```
target = implementasi_array_3d[2][1:][0]
print(target)
```

```
[3 0]
```

Gambar 13: Kode untuk melakukan slicing

C.1.13 Mengonversi tipe data

Untuk melakukan konversi tipe data pada array, dapat menggunakan fungsi `astype()`. Gambar 14 adalah contoh kode untuk mengonversi tipe data array.



Gambar 14: Kode untuk mengonversi tipe data array

C.1.14 Reshape ke dimensi lebih tinggi

Untuk mengubah bentuk array menjadi dimensi yang lebih tinggi, dapat menggunakan fungsi `reshape()`. Gambar 15 adalah contoh kode untuk melakukan reshape ke dimensi lebih tinggi.

```python
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
print(arr)
print('Shape array: ', arr.shape, '\n')

new_arr_2d = arr.reshape(3, 4)
print(new_arr_2d)
print('Shape array 2D: ', new_arr_2d.shape, '\n')

new_arr_3d = arr.reshape(2, 3, 2)

print(new_arr_3d)
print('Shape array 3D: ', new_arr_3d.shape, '\n')
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
Shape array:  (12,)

[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Shape array 2D:  (3, 4)

[[[ 1  2]
  [ 3  4]
  [ 5  6]]

 [[ 7  8]
  [ 9 10]
  [11 12]]]
Shape array 3D:  (2, 3, 2)
```

Gambar 15: Kode untuk melakukan reshape ke dimensi lebih tinggi

Berdasarkan gambar 15, array yang awalnya memiliki dimesnsi 1 (shape 12,) diubah menjadi array dengan dimensi 2 (shape 3, 4) dan dimensi 3 (shape 3, 2, 2). Namun, jumlah elemen pada array tetap sama yaitu 12 elemen.

C.1.15 Reshape ke dimensi 1

Untuk mengubah bentuk array menjadi dimensi 1, dapat menggunakan fungsi `reshape()` dengan memasukkan argumen -1. Gambar 16 adalah contoh kode untuk melakukan reshape ke dimensi 1.

```
    #3d ke 1 d
    arr = np. array ([
    [[1, 2, 3], [4, 5, 6]],
    [[10, 11, 12], [13, 14, 15]]
    ])

    print(arr)
    print('sesudah di reshape')
    new_arr = arr.reshape(-1)

    print(new_arr)
```

```
[[[ 1  2  3]
  [ 4  5  6]]

 [[10 11 12]
  [13 14 15]]]
sesudah di reshape
[ 1  2  3  4  5  6 10 11 12 13 14 15]
```

Gambar 16: Kode untuk melakukan reshape ke dimensi 1

C.1.16 Perbedaan fungsi len dan atribut shape

Len hanya dapat digunakan untuk melihat jumlah elemen pada sumbu 0 (axis 0) array, sedangkan shape dapat digunakan untuk melihat jumlah elemen pada semua sumbu array. Gambar 17 adalah contoh kode untuk melihat perbedaan fungsi len dan atribut shape.

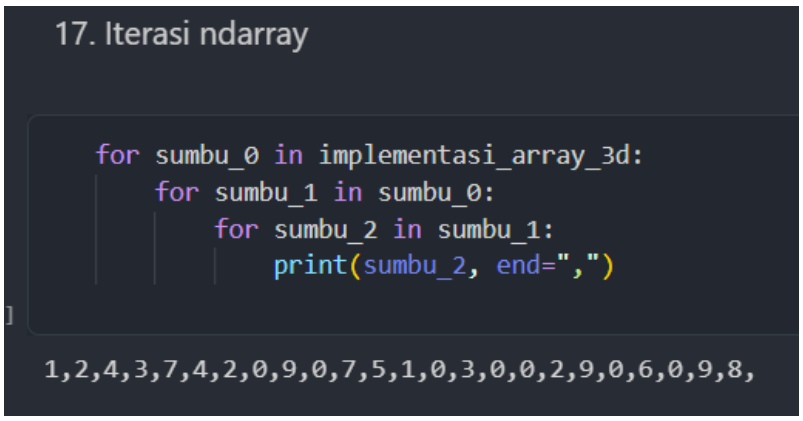
```
print(len(implementasi_array_3d))
print(implementasi_array_3d.shape)
```

```
4
(4, 3, 2)
```

Gambar 17: Kode untuk melihat perbedaan fungsi len dan atribut shape

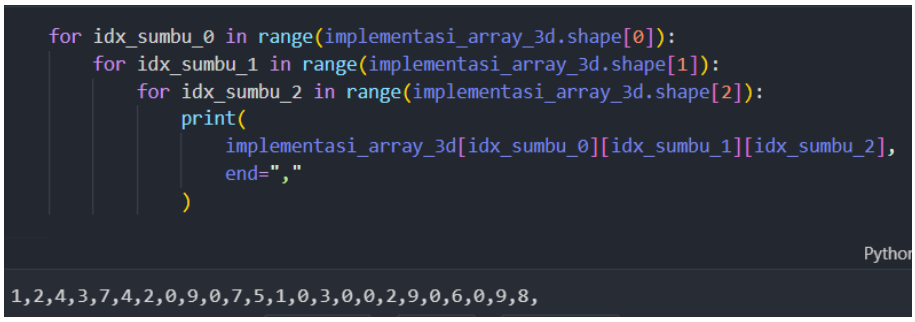
C.1.17 Iterasi array

Iterasi array adalah proses untuk mengakses setiap elemen pada array. Gambar 18 adalah contoh kode untuk melakukan iterasi array.



Gambar 18: Kode untuk melakukan iterasi array

Iterasi array juga dapat dilakukan dengan indeks. Gambar 19 adalah contoh kode untuk melakukan iterasi array dengan indeks.



Gambar 19: Kode untuk melakukan iterasi array dengan indeks

C.1.18 Join pada array dimensi 1

Menggabungkan dua array 1 dimensi dapat dilakukan dengan fungsi `np.concatenate()`. Gambar 20 adalah contoh kode untuk melakukan join pada array dimensi 1.

```
arr1 = np.array([1,2,3])
arr2 = np.array([4,5,6])
arr3 = np.concatenate((arr1, arr2), axis=0)
print(arr3)
```

```
[1 2 3 4 5 6]
```

Gambar 20: Kode untuk melakukan join pada array dimensi 1

C.1.19 Join pada array dimensi 2

Untuk menggabungkan dua array 2 dimensi, dapat menggunakan fungsi `np.concatenate()` dengan memasukkan argumen axis (yaitu 1). Gambar 21 adalah contoh kode untuk melakukan join pada array dimensi 2.

```
arr1 = np.array([[1,2,3], [4,5,6]])
arr2 = np.array([[11,12,13], [14,15,16]])

#menambah di sumbu 1, kolomnya
arr3 = np.concatenate((arr1, arr2), axis=1)
print(arr3)
```

```
[[ 1  2  3 11 12 13]
 [ 4  5  6 14 15 16]]
```

Gambar 21: Kode untuk melakukan join pada array dimensi 2

Jika akan menambahkan baris pada array 2 dimensi, maka argumen axis diisi dengan 0. Gambar 22 adalah contoh kode untuk menambahkan baris pada array 2 dimensi.

```python
arr3 = np.concatenate((arr1, arr2), axis=0)
print(arr3)
```

```
[[ 1  2  3]
 [ 4  5  6]
 [11 12 13]
 [14 15 16]]
```

Gambar 22: Kode untuk menambahkan baris pada array 2 dimensi

C.1.20 Array Split

Split pada array dapat dilakukan dengan fungsi `np.array_split()`. Gambar 23 adalah contoh kode untuk melakukan split pada array.

```python
arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
#dipisah jadi 5 bagian array
newarr = np.array_split(arr, 5)
print('sebelum')
print(arr)
print('sesudah')
print(newarr)
narr = newarr[2]
print(narr)
```

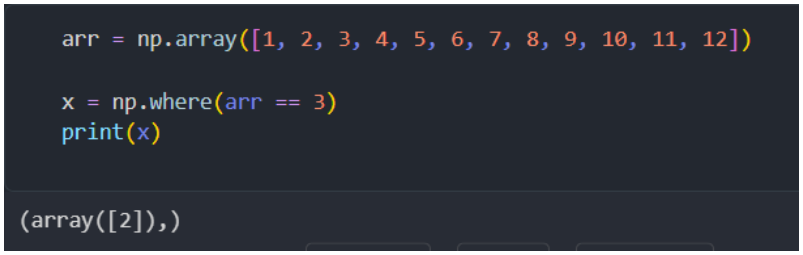
Python

```
sebelum
[ 1  2  3  4  5  6  7  8  9 10 11 12]
sesudah
[array([1, 2, 3]), array([4, 5, 6]), array([7, 8]), array([ 9, 10]), array([11, 12]
[7 8]
```

Gambar 23: Kode untuk melakukan split pada array

C.1.21 Array Search

Array search dapat dilakukan dengan fungsi `np.where()`. Gambar 24 adalah contoh kode untuk melakukan array search.

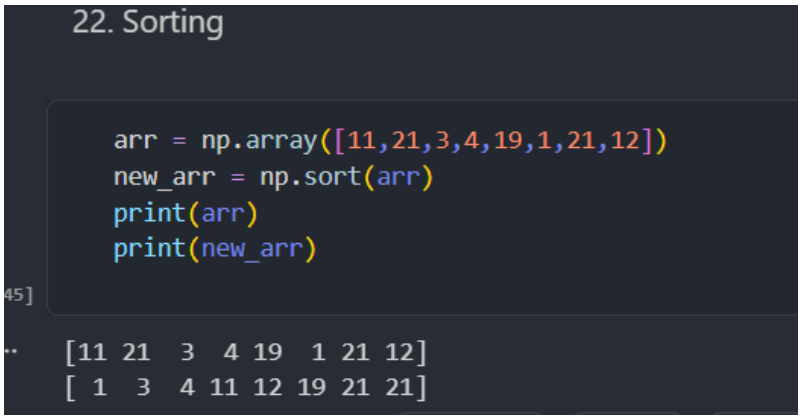


Gambar 24: Kode untuk melakukan array search

Berdasarkan gambar 24, elemen dengan nilai 3 dicari dan ditemukan dengan indeks 2 pada array. Sehingga output yang dihasilkan adalah (array([2]),).

C.1.22 Sorting pada array

Sorting pada array dapat dilakukan dengan fungsi `np.sort()`. Gambar 25 adalah contoh kode untuk melakukan sorting pada array.



Gambar 25: Kode untuk melakukan sorting pada array

C.1.23 NumPy Random

Numpy Random adalah modul di Numpy yang digunakan untuk menghasilkan bilangan acak. Gambar 26 adalah contoh kode untuk menggunakan Numpy Random.

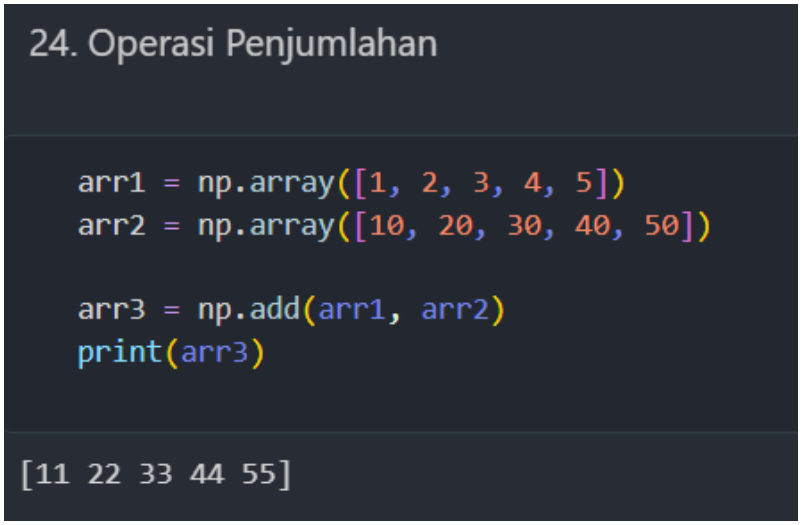


Gambar 26: Kode untuk menggunakan Numpy Random

Berdasarkan kode tersebut, nilai random akan dihasilkan dengan shape dimensi 1 yang berisi 2 elemen.

C.1.24 Operasi Penjumlahan

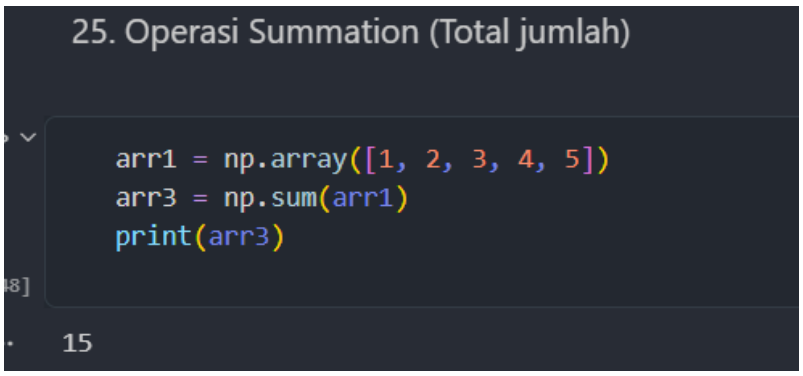
Numpy memiliki fungsi untuk melakukan operasi penjumlahan pada array. Gambar 27 adalah contoh kode untuk melakukan operasi penjumlahan pada array.



Gambar 27: Kode untuk melakukan operasi penjumlahan pada array

C.1.25 Operasi Summation

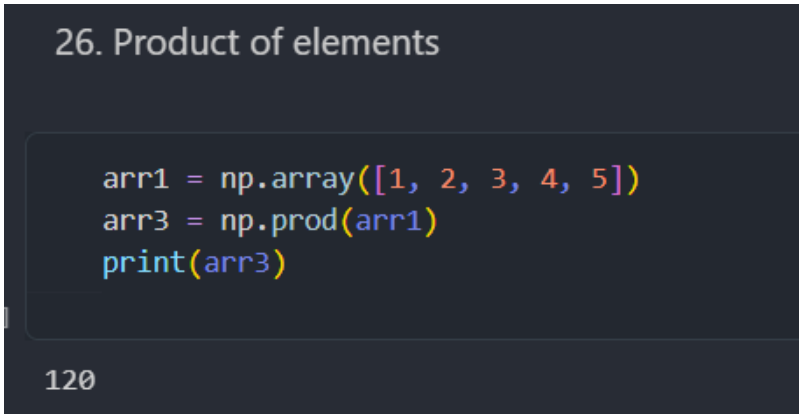
Numpy memiliki fungsi untuk melakukan operasi penjumlahan pada array. Gambar 28 adalah contoh kode untuk melakukan operasi penjumlahan pada array.



Gambar 28: Kode untuk melakukan operasi summation pada array

C.1.26 Product of Elements

Untuk menghitung semua perkalian elemen pada array, dapat menggunakan fungsi `np.prod()`. Gambar 29 adalah contoh kode untuk menghitung semua perkalian elemen pada array.



Gambar 29: Kode untuk menghitung semua perkalian elemen pada array

C.2 Dataframe Pandas

C.2.1 Install Pandas

Pandas digunakan untuk mengolah data dalam bentuk tabel. Untuk menginstall library Pandas, dapat menggunakan perintah berikut.

```
!pip install pandas
```

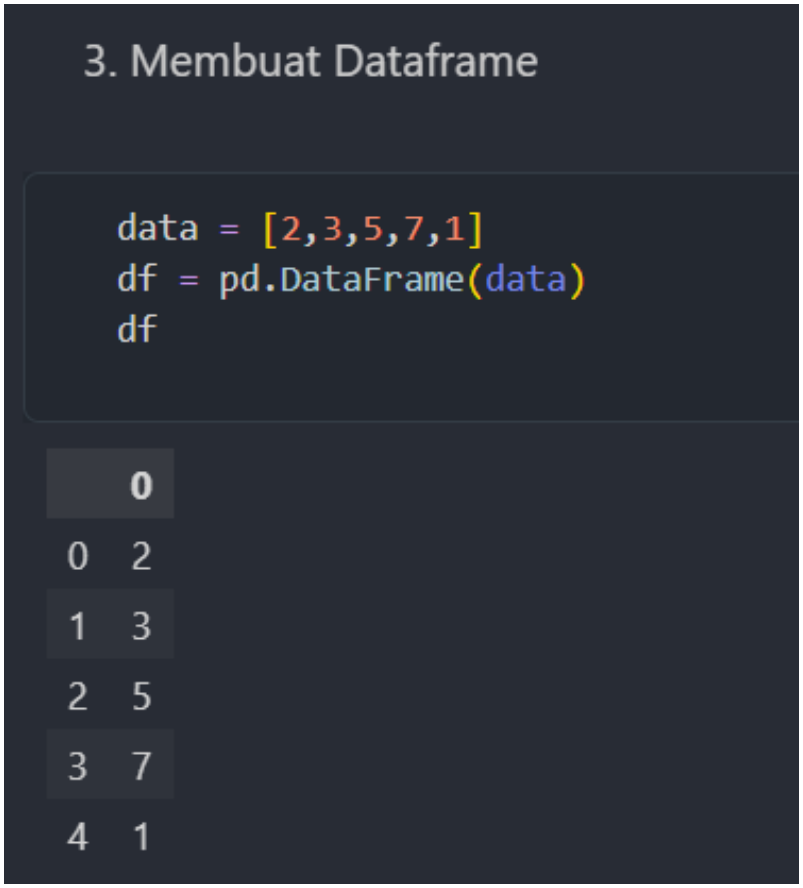
C.2.2 Import Library Pandas

Untuk menggunakan library Pandas, perlu dilakukan import library terlebih dahulu. Berikut perintah untuk mengimpor library Pandas.

```
import pandas as pd
import numpy as np
```

C.2.3 Membuat Dataframe

DataFrame dibuat dengan memberikan sebuah list python sebagai input dari fungsi `pd.DataFrame()`. Gambar 30 adalah contoh kode untuk membuat DataFrame.



Gambar 30: Kode untuk membuat DataFrame

Dapat juga menambahkan parameter `columns` untuk memberikan nama kolom pada DataFrame dan parameter indeks untuk memberikan label indeks custom. Gambar 31 adalah contoh kode untuk membuat DataFrame dengan nama kolom.

```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'], index=['satu', 'dua', 'tiga'])
df
```

	Nama	Umur
satu	Adi	34
dua	Maya	23
tiga	Joko	20

Gambar 31: Kode untuk membuat DataFrame dengan nama kolom

C.2.4 Menambahkan kolom

Untuk menambahkan kolom pada DataFrame, dapat menggunakan operator `df['namakolom]` dengan memasukkan nama kolom baru dan memberikan nilai pada kolom tersebut. Gambar 32 adalah contoh kode untuk menambahkan kolom pada DataFrame.

4. Menambahkan Kolom

```
data = {'Nama ': ['Jack', 'Joko'], 'Umur' : [20, 23]}
df = pd.DataFrame(data)
df['Alamat'] = ['Surabaya', 'Yogyakarta']
df
```

	Nama	Umur	Alamat
0	Jack	20	Surabaya
1	Joko	23	Yogyakarta

Gambar 32: Kode untuk menambahkan kolom pada DataFrame

Selain menggunakan operator `df['namakolom']`, dapat juga menggunakan fungsi `df.insert()` untuk menambahkan kolom baru. Kolom juga dapat ditambahkan pada posisi tertentu dengan menambahakan parameter loc, column, value.



Gambar 33: Kode untuk menambahkan kolom pada DataFrame dengan fungsi insert()

C.2.5 Menambahkan multikolom

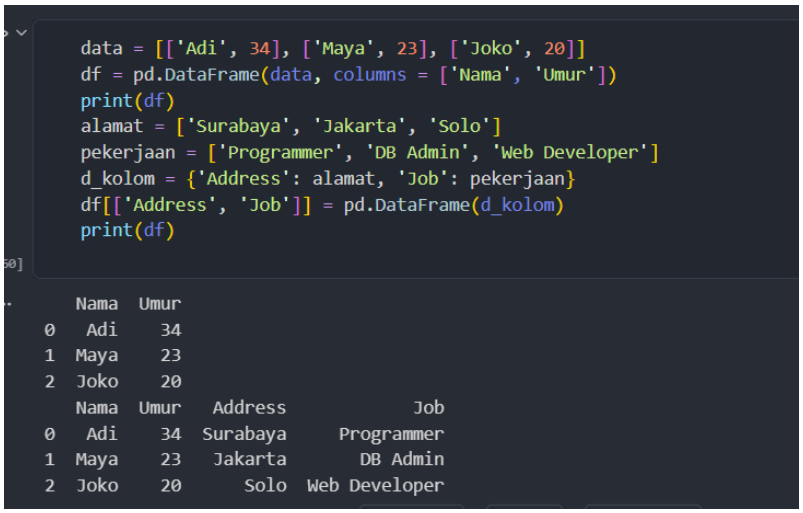
Untuk menambahkan multi kolom dapat menggunakan fungsi `df.assign()` dengan memasukkan nama kolom baru dan nilai pada kolom tersebut. Gambar 34 adalah contoh kode untuk menambahkan multi kolom pada DataFrame.



Gambar 34: Kode untuk menambahkan multi kolom pada DataFrame

Selain itu, dapat juga menggunakan sebuah dictionary untuk menambahkan

multi kolom pada DataFrame. Gambar 35 adalah contoh kode untuk menambahkan multi kolom pada DataFrame dengan menggunakan dictionary.



```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'])
print(df)
alamat = ['Surabaya', 'Jakarta', 'Solo']
pekerjaan = ['Programmer', 'DB Admin', 'Web Developer']
d_kolom = {'Address': alamat, 'Job': pekerjaan}
df[['Address', 'Job']] = pd.DataFrame(d_kolom)
print(df)
```

```
   Nama  Umur
0   Adi    34
1  Maya    23
2  Joko    20
   Nama  Umur   Address           Job
0   Adi    34  Surabaya     Programmer
1  Maya    23   Jakarta       DB Admin
2  Joko    20      Solo  Web Developer
```

Gambar 35: Kode untuk menambahkan multi kolom pada DataFrame dengan menggunakan dictionary

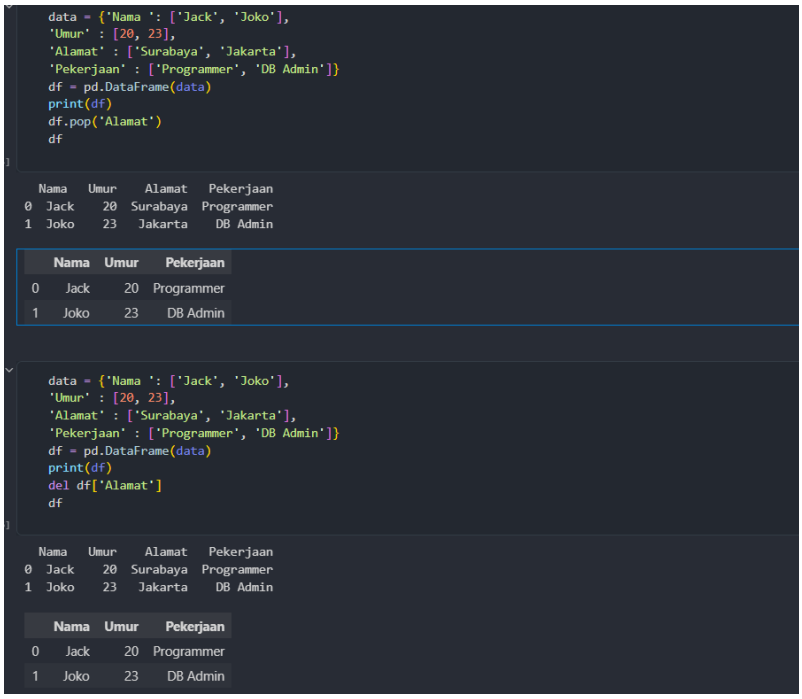
C.2.6 Menghapus kolom

Untuk menghapus kolom pada DataFrame, dapat menggunakan fungsi `df.drop()` dengan memasukkan nama kolom yang ingin dihapus. Gambar 36 adalah contoh kode untuk menghapus kolom pada DataFrame.



Gambar 36: Kode untuk menghapus kolom pada DataFrame

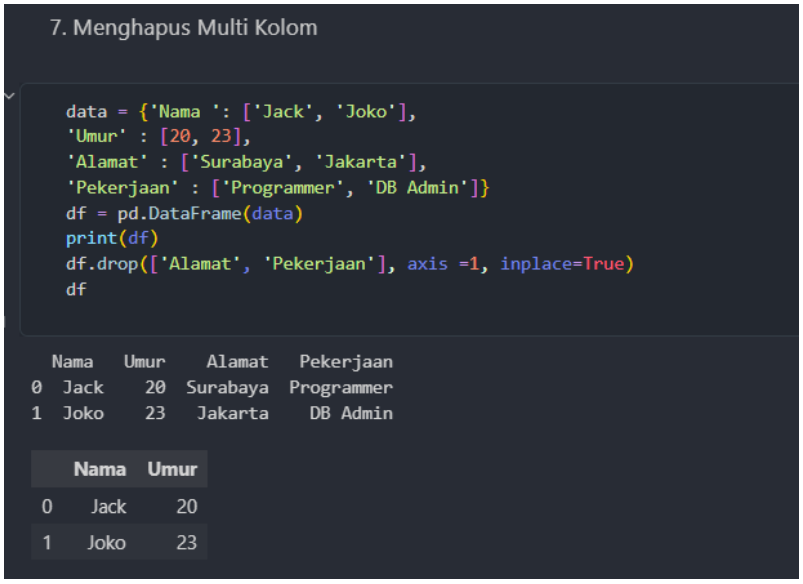
Selain itu, dapat juga menggunakan fungsi `df.pop()` dan fungsi `del df['nama kolom]` menghapus kolom pada DataFrame. Gambar 37 adalah contoh kodenya.



Gambar 37: Kode untuk menghapus kolom pada DataFrame dengan fungsi pop()

C.2.7 Menghapus Multi Kolom

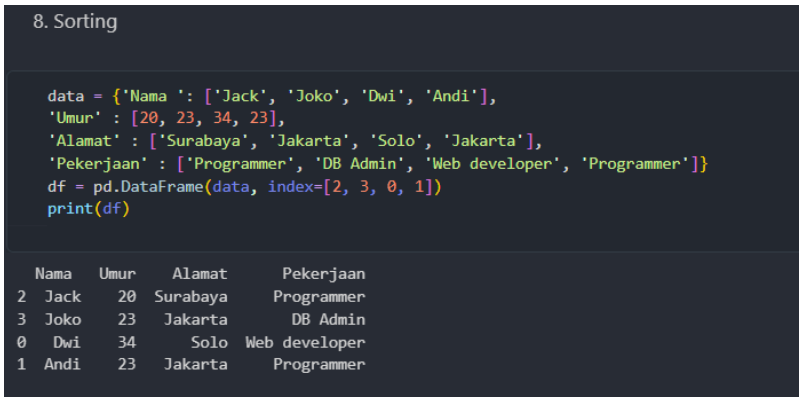
Untuk menghapus multi kolom pada DataFrame, dapat menggunakan fungsi `df.drop()` dengan memasukkan list nama kolom yang ingin dihapus. Gambar 38 adalah contoh kode untuk menghapus multi kolom pada DataFrame.



Gambar 38: Kode untuk menghapus multi kolom pada DataFrame

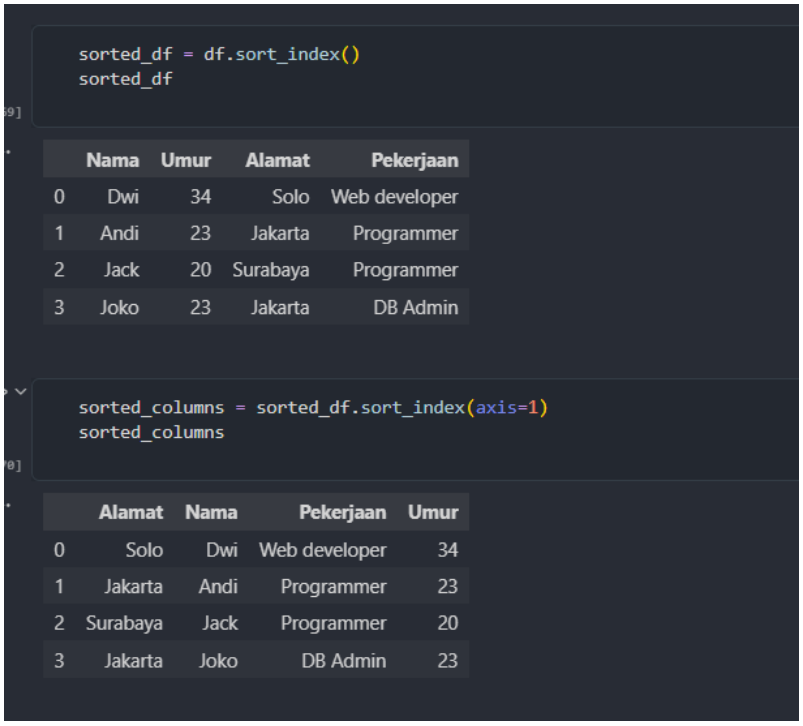
C.2.8 Sorting DataFrame

Mengurutkan data berdasarkan indeks dapat dilakukan dengan fungsi `df.sort_index()`. Gambar 39 adalah contoh kode untuk mengurutkan data berdasarkan indeks.



Gambar 39: Kode untuk mengurutkan data berdasarkan indeks

Dapat juga menggunakan fungsi `df.sort_index()` untuk mengurutkan data berdasarkan kolom tertentu. Serta parameter axis=1 untuk mengurutkan kolom berdasarkan indeks axis. Gambar 40 adalah contoh kode untuk mengurutkan data berdasarkan kolom tertentu.



```
sorted_df = df.sort_index()
sorted_df
```

	Nama	Umur	Alamat	Pekerjaan
0	Dwi	34	Solo	Web developer
1	Andi	23	Jakarta	Programmer
2	Jack	20	Surabaya	Programmer
3	Joko	23	Jakarta	DB Admin

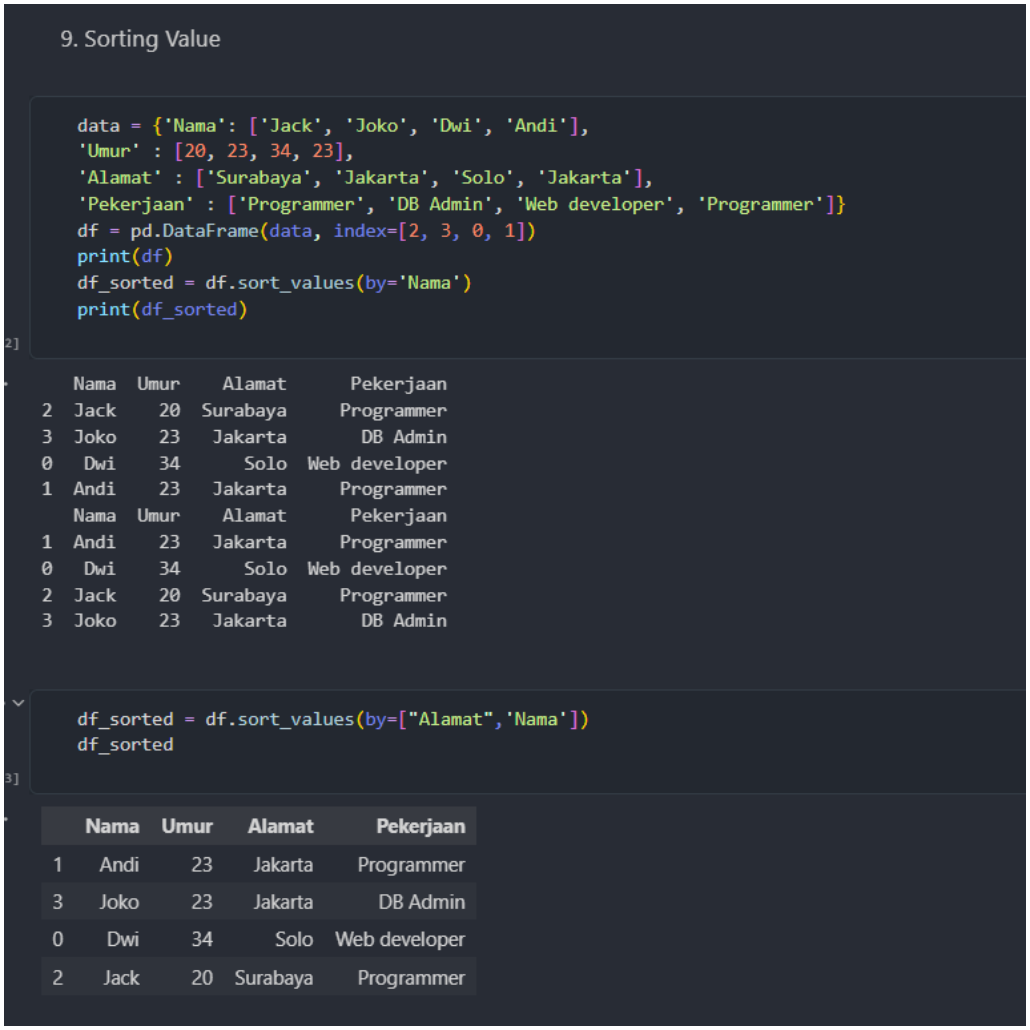
```
sorted_columns = sorted_df.sort_index(axis=1)
sorted_columns
```

	Alamat	Nama	Pekerjaan	Umur
0	Solo	Dwi	Web developer	34
1	Jakarta	Andi	Programmer	23
2	Surabaya	Jack	Programmer	20
3	Jakarta	Joko	DB Admin	23

Gambar 40: Kode untuk mengurutkan data berdasarkan kolom tertentu

C.2.9 Sorting Value

Untuk melakukan sorting berdasarkan nilai pada kolom tertentu, dapat menggunakan fungsi `df.sort_values()`. Gambar 41 adalah contoh kode untuk melakukan sorting berdasarkan nilai pada kolom tertentu.



Gambar 41: Kode untuk melakukan sorting berdasarkan nilai pada kolom tertentu

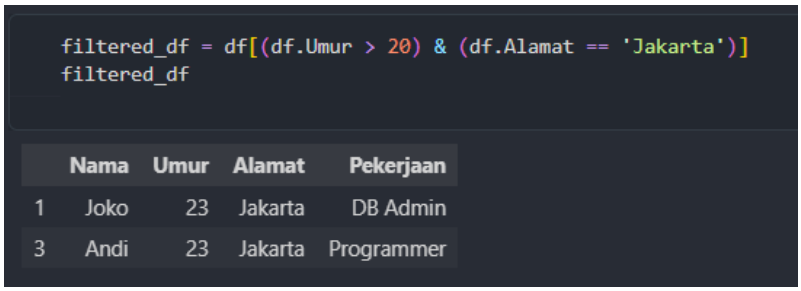
C.2.10 Filtering DataFrame

Filtering dataset dilakukan dengan memberikan kondisi logika pada DataFrame. Gambar 42 adalah contoh kode untuk melakukan filtering DataFrame.



Gambar 42: Kode untuk melakukan filtering DataFrame

Selain itu, dapat juga melakukan filter data dengan logika lebih dari satu kondisi. Gambar 43 adalah contoh kode untuk melakukan filtering DataFrame dengan logika lebih dari satu kondisi.



Gambar 43: Kode untuk melakukan filtering DataFrame dengan logika lebih dari satu kondisi

Bisa juga menggunakan fungsi `df.query()` untuk melakukan filtering DataFrame. Gambar 44 adalah contoh kode untuk melakukan filtering DataFrame dengan fungsi query().

```
filtered = df.query("Nama=='Joko' | Nama=='Andi'")
filtered
```

	Nama	Umur	Alamat	Pekerjaan
1	Joko	23	Jakarta	DB Admin
3	Andi	23	Jakarta	Programmer

Gambar 44: Kode untuk melakukan filtering DataFrame dengan fungsi query()

C.2.11 Grouping DataFrame

Grouping DataFrame dilakukan dengan fungsi `df.groupby()`. Gambar 45 adalah contoh kode untuk melakukan grouping DataFrame.

11. Grouping

```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
'Umur' : [20, 23, 34, 23],
'Alamat' : ['Surabaya', 'Jakarta', 'solo', 'Jakarta'],
'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer'],
'Gaji' : [2000, 3000, 2400, 5000]}
df = pd.DataFrame(data)
df
```

	Nama	Umur	Alamat	Pekerjaan	Gaji
0	Jack	20	Surabaya	Programmer	2000
1	Joko	23	Jakarta	DB Admin	3000
2	Dwi	34	solo	Web developer	2400
3	Andi	23	Jakarta	Programmer	5000

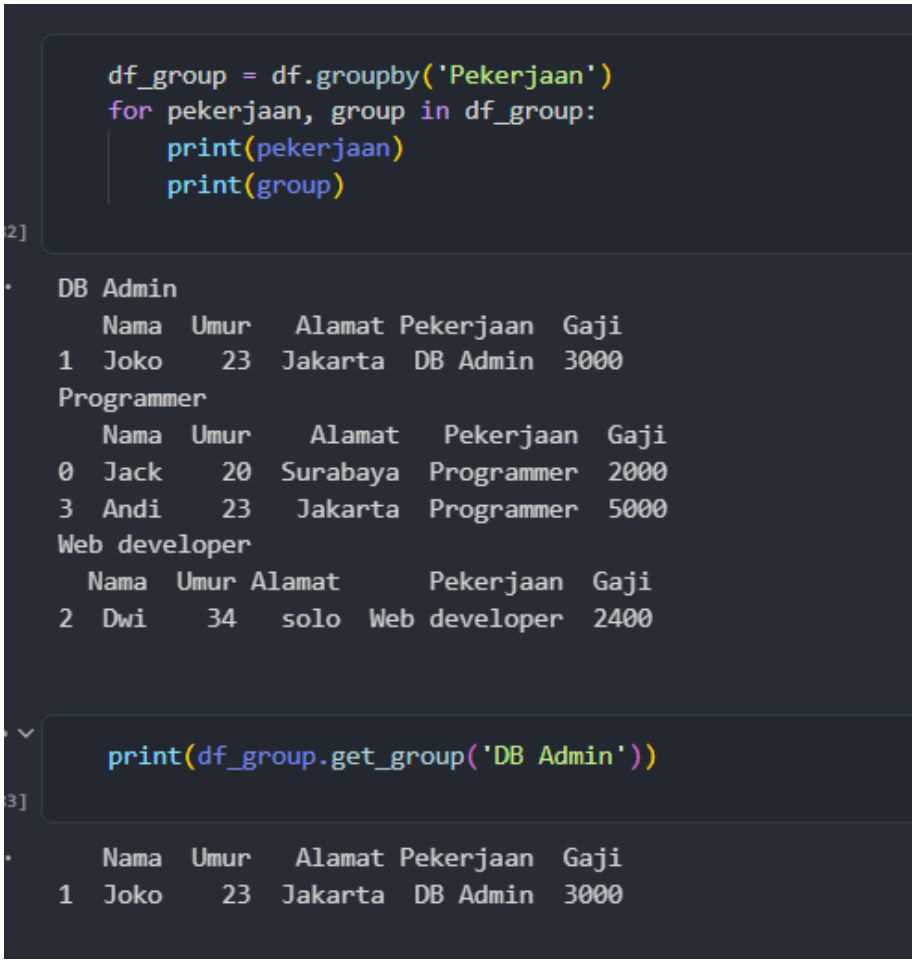
```
df.groupby('Pekerjaan').groups
```

```
{'DB Admin': [1], 'Programmer': [0, 3], 'Web developer': [2]}
```

Gambar 45: Kode untuk melakukan grouping DataFrame

Objek hasil grouping diiterasi untuk menampilkan setiap kelompok data secara terpisah. Fungsi `get_group()` digunakan untuk menampilkan data pada kelompok tertentu. Gambar 46 adalah contoh kode untuk menampilkan data pada kelompok

tertentu.



Gambar 46: Kode untuk menampilkan data pada kelompok tertentu

C.2.12 Aggregation DataFrame

Operasi agregate dilakukan dengan menerapkan fungsi seperti mean, sum, count, min, max pada setiap kelompok data. Gambar 47 adalah contoh kode untuk melakukan agregasi DataFrame.

12. Aggregation

```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
'Umur' : [20, 23, 34, 23],
'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer'],
'Gaji' : [2000, 3000, 2400, 5000]}
df = pd.DataFrame(data)
print('sebelum di kelompokkan')
print(df)
df_group = df.groupby('Pekerjaan').mean(numeric_only=True)
print('setelah dikelompokkan')
df_group
```

```
sebelum di kelompokkan
   Nama  Umur    Alamat      Pekerjaan  Gaji
0  Jack    20  Surabaya     Programmer  2000
1  Joko    23   Jakarta       DB Admin  3000
2   Dwi    34      Solo  Web developer  2400
3  Andi    23   Jakarta     Programmer  5000
setelah dikelompokkan
```

Pekerjaan	Umur	Gaji
DB Admin	23.0	3000.0
Programmer	21.5	3500.0
Web developer	34.0	2400.0

Gambar 47: Kode untuk melakukan agregasi DataFrame

Agregasi juga dapat dilakukan dengan menerapkan lebih dari satu fungsi pada setiap kelompok data. Gambar 48 adalah contoh kode untuk melakukan agregasi DataFrame dengan lebih dari satu fungsi.



Gambar 48: Kode untuk melakukan agregasi DataFrame dengan lebih dari satu fungsi

C.2.13 Merge

Penggabungan dua DataFrame dapat dilakukan dengan fungsi `pd.merge()`. Gambar 49 adalah contoh kode untuk melakukan merge pada dua DataFrame.



```
kiri = pd.DataFrame({'Id': [2, 3, 1],
'Nama' : ['Andi', 'Joko', 'Budi']})
kanan = pd.DataFrame({'Id':[1, 2,3],
'Nama' : ['Mike', 'Jack', 'Jane']})
gabung = pd.merge(kiri, kanan, on='Id', sort=True)
gabung
```

	Id	Nama_x	Nama_y
0	1	Budi	Mike
1	2	Andi	Jack
2	3	Joko	Jane

```
karyawan = pd.DataFrame({
'Nama': ['Budi', 'Joko', 'Maya'],
'Alamat': ['Jakarta', 'Surabaya', 'Medan'],
'Id Pekerjaan' : [1,7,2]
})

pekerjaan = pd.DataFrame({
'Id Pekerjaan': [2,1,4,3],
'Nama': ['Programmer', 'Data Engineer', 'Manager', 'Web Developer'],
'Gaji' : [5000, 4000, 3000, 6000]
})

print(karyawan)
print(pekerjaan)
```

```
   Nama    Alamat  Id Pekerjaan
0  Budi   Jakarta             1
1  Joko  Surabaya             7
2  Maya     Medan             2
   Id Pekerjaan           Nama  Gaji
0             2     Programmer  5000
1             1  Data Engineer  4000
2             4        Manager  3000
3             3  Web Developer  6000
```

Gambar 49: Kode untuk melakukan merge pada dua DataFrame

Selain itu, merge juga dapat dilakukan dengan fungsi merge() yang diberi parameter how=’left’, ’right’, ’outer’, atau ’inner’ untuk menentukan jenis penggabungan. Gambar 50 adalah contoh kode untuk melakukan merge dengan parameter how.

```
gabung_merge = pd.merge(karyawan, pekerjaan, on='Id Pekerjaan',
how='left', sort=True)
gabung_merge
```

	Nama_x	Alamat	Id Pekerjaan	Nama_y	Gaji
0	Budi	Jakarta	1	Data Engineer	4000.0
1	Maya	Medan	2	Programmer	5000.0
2	Joko	Surabaya	7	NaN	NaN

Gambar 50: Kode untuk melakukan merge dengan parameter how='left'

C.2.14 Join

Join dapat dilakukan dengan menggunakan merge() pada key column yang sama, di mana parameter how='left' memastikan seluruh data dari tabel kiri ditampilkan, sedangkan suffixes digunakan untuk membedakan nama kolom yang berasal dari kedua DataFrame.

14. Join

```
#menggunakan suffixes
gabung_merge = pd.merge(karyawan, pekerjaan, on='Id Pekerjaan',
how='left', sort=True, suffixes=('_pegawai', '_pekerjaan'))
gabung_merge
```

	Nama_pegawai	Alamat	Id Pekerjaan	Nama_pekerjaan	Gaji
0	Budi	Jakarta	1	Data Engineer	4000.0
1	Maya	Medan	2	Programmer	5000.0
2	Joko	Surabaya	7	NaN	NaN

Gambar 51: Kode untuk melakukan join pada dua DataFrame

Join juga dapat dilakukan dengan fungsi join() yang menggabungkan DataFrame berdasarkan indeks. Gambar 52 adalah contoh kode untuk melakukan join dengan fungsi join().

```
#versi 1. hasil tidak sama dengan merge()
gabung_join = karyawan. join(pekerjaan, how='left', rsuffix='_pekerjaan')
gabung_join
```

	Nama	Alamat	Id Pekerjaan	Id Pekerjaan_pekerjaan	Nama_pekerjaan	Gaji
0	Budi	Jakarta	1	2	Programmer	5000
1	Joko	Surabaya	7	1	Data Engineer	4000
2	Maya	Medan	2	4	Manager	3000

Gambar 52: Kode untuk melakukan join dengan fungsi join()

C.2.15 Concatenate

Concatenate dapat dilakukan dengan fungsi `pd.concat()` pada axis=0, sehingga DataFrame kedua ditambahkan sebagai baris baru di bawah DataFrame pertama. Parameter ignore_index=True digunakan agar indeks diurutkan ulang secara otomatis.

```
# menjadi baris baru. axis=0
kiri = pd.DataFrame({
'Nama': ['Budi', 'Joko', 'Maya'],
'Alamat' : ['Jakarta', 'Surabay', 'Medan']
})

kanan = kiri = pd.DataFrame({
'Nama': ['Jane', 'mike', 'dave'],
'Alamat' : ['Semarang', 'Yogya', 'solo']
})

gabung_concat = pd.concat([kiri, kanan], axis=0, ignore_index=True)
gabung_concat
```

	Nama	Alamat
0	Jane	Semarang
1	mike	Yogya
2	dave	solo
3	Jane	Semarang
4	mike	Yogya
5	dave	solo

Gambar 53: Kode untuk melakukan concatenate pada dua DataFrame

Dengan menambahkan parameter axis=0 dan parameter join='outer' dapat menggabungkan semua kolom dari kedua dataframe. Kolom yang tidak dimiliki oleh salah satu DataFrame akan berisi nilai NaN, sementara parameter ignore_index=True mengatur ulang indeks agar berurutan.

```
# menjadi baris baru. axis=0
kiri = pd.DataFrame({
'Nama': ['Budi', 'Joko', 'Maya'],
'Alamat' : ['Jakarta', 'Surabay', 'Medan']
})

kanan = pd.DataFrame({
'Nama': ['Jane', 'mike', 'dave'],
'Gaji' : [5000, 4000, 3000]
})

gabung_concat = pd.concat([kiri, kanan], axis=0, ignore_index=True, join='outer')
gabung_concat
```

	Nama	Alamat	Gaji
0	Budi	Jakarta	NaN
1	Joko	Surabay	NaN
2	Maya	Medan	NaN
3	Jane	NaN	5000.0
4	mike	NaN	4000.0
5	dave	NaN	3000.0

Gambar 54: Kode untuk melakukan concatenate pada dua DataFrame dengan join='outer'

Jika menggunakan axis=1, penggabungan dilakukan secara horizontal sehingga kolom baru ditambahkan di samping DataFrame awal.

```python
# menjadi baris baru. axis=0
kiri = pd.DataFrame({
'Nama': ['Budi', 'Joko', 'Maya'],
'Alamat' : ['Jakarta', 'Surabay', 'Medan']
})

kanan = pd.DataFrame({
'Nama': ['Jane', 'mike', 'dave'],
'Gaji' : [5000, 4000, 3000]
})

gabung_concat = pd.concat([kiri, kanan], axis=1, ignore_index=True, join='outer')
gabung_concat
```

	0	1	2	3
0	Budi	Jakarta	Jane	5000
1	Joko	Surabay	mike	4000
2	Maya	Medan	dave	3000

Gambar 55: Kode untuk melakukan concatenate pada dua DataFrame dengan axis=1

C.2.16 Reset Indeks

Reset indeks dapat dilakukan dengan menggunakan pd.concat() dan menetapkan ignore_index=True. Gambar 56 adalah contoh kode untuk melakukan reset indeks pada DataFrame.

16. Reset Indeks

```
kiri = pd.DataFrame({
'Nama': ['Budi', 'Joko', 'Maya'],
'Alamat' : ['Jakarta', 'Surabay', 'Medan']
})

kanan = pd.DataFrame({
'Nama': ['Jane', 'mike', 'dave'],
'Alamat' : ['Semarang', 'Yogya', 'Solo']})

gabung_append = pd.concat([kiri, kanan], ignore_index=True)
gabung_append
```

	Nama	Alamat
0	Budi	Jakarta
1	Joko	Surabay
2	Maya	Medan
3	Jane	Semarang
4	mike	Yogya
5	dave	Solo

Gambar 56: Kode untuk melakukan reset indeks pada DataFrame

Reset indeks juga dapat dilakukan dengan fungsi reset_index(drop=True), yang menghapus indeks lama dan menggantinya dengan indeks baru yang berurutan. Parameter drop=True memastikan indeks sebelumnya tidak ikut disimpan sebagai kolom baru. Gambar 57 adalah contoh kode untuk melakukan reset indeks dengan fungsi reset_index().

```
new = gabung_append.reset_index(drop=True)
new
```

	Nama	Alamat
0	Budi	Jakarta
1	Joko	Surabay
2	Maya	Medan
3	Jane	Semarang
4	mike	Yogya
5	dave	Solo

Gambar 57: Kode untuk melakukan reset indeks pada DataFrame dengan fungsi reset_index()

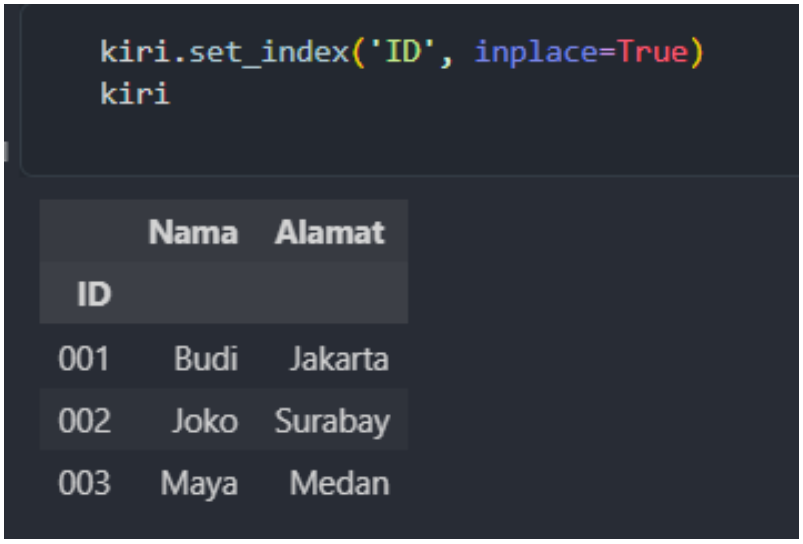
C.2.17 Set Indeks

Set indeks dapat dilakukan dengan memilih baris tertentu sebagai referensi indeks baru. Pada Gambar 1.4.73, nilai pada baris ke-2 ditampilkan karena DataFrame sebelumnya telah atau akan diubah indeksnya menggunakan nilai kolom tertentu melalui set_index(), sehingga baris tersebut ditampilkan sesuai indeks barunya.



Gambar 58: Kode untuk melakukan set indeks pada DataFrame

Set indeks juga dapat dilakukan dengan fungsi set_index() yang menetapkan kolom ID sebagai indeks baru DataFrame.



Gambar 59: Kode untuk melakukan set indeks pada DataFrame dengan fungsi set_index()

C.2.18 Pivot Table

Pivot tabel dibuat menggunakan pd.pivot_table() dengan menetapkan kolom Gender dan Pekerjaan sebagai pembentuk kolom baru. Agregasi dilakukan melalui parameter aggfunc, untuk menjumlahkan nilai Gaji dan menghitung jumlah data berdasarkan Pekerjaan seperti pada Gambar 60.

18.Pivot Tabel

```
karyawan = pd.DataFrame({
'Emp_id':['001','002','003','004','005','006'],
'Nama': ['Budi', 'Joko', 'Maya', 'Andi', 'Dewi', 'Dian'],
'Gender':['Pria', 'Pria', 'Perempuan', 'Pria', 'Perempuan', 'Perempuan'],
'Pekerjaan': ['Programmer', 'Data Engineer', 'Programmer', 'Manager',
'Manager', 'Manager' ],
'Gaji':[5000, 4000, 5000, 6000, 8000, 6000]})
karyawan
```

5]

	Emp_id	Nama	Gender	Pekerjaan	Gaji
0	001	Budi	Pria	Programmer	5000
1	002	Joko	Pria	Data Engineer	4000
2	003	Maya	Perempuan	Programmer	5000
3	004	Andi	Pria	Manager	6000
4	005	Dewi	Perempuan	Manager	8000
5	006	Dian	Perempuan	Manager	6000

Gambar 60: Kode untuk membuat pivot table pada DataFrame

Untuk menghitung sum, mean, dll dapat menggunakan parameter aggfunc dengan np.sum. Gambar 61 adalah contoh kode untuk membuat pivot table dengan parameter aggfunc.

```
import numpy as np
pivot_table = pd.pivot_table(karyawan, columns=['Gender', 'Pekerjaan'], aggfunc={'Gaji': np.sum, 'Pekerjaan':'count'
pivot_table
```

Python

Gender	Perempuan		Pria		
Pekerjaan	Manager	Programmer	Data Engineer	Manager	Programmer
Gaji	14000	5000	4000	6000	5000
Pekerjaan	2	1	1	1	1

Gambar 61: Kode untuk membuat pivot table pada DataFrame dengan parameter aggfunc

C.2.19 Melt

Melt digunakan untuk mengubah format DataFrame dari lebar menjadi panjang. Gambar 62 adalah contoh kode untuk melakukan melt pada DataFrame.

19. Melt

```
mahasiswa = pd.DataFrame({
'Mhs_id' :['001','002','003'],
'Nama': ['Budi', 'Joko', 'Maya'],
'Bhs Inggris': [100, 90, 98],
'Matematika': [90, 87, 72],
'Ekonomi':[90, 80, 78]})
mahasiswa
```

Python

	Mhs_id	Nama	Bhs Inggris	Matematika	Ekonomi
0	001	Budi	100	90	90
1	002	Joko	90	87	80
2	003	Maya	98	72	78

```
melt_table = pd.melt(mahasiswa, id_vars=['Mhs_id', 'Nama'], value_vars=['Bhs Inggris', 'Matematika', 'Ekonomi'])
melt_table
```

Python

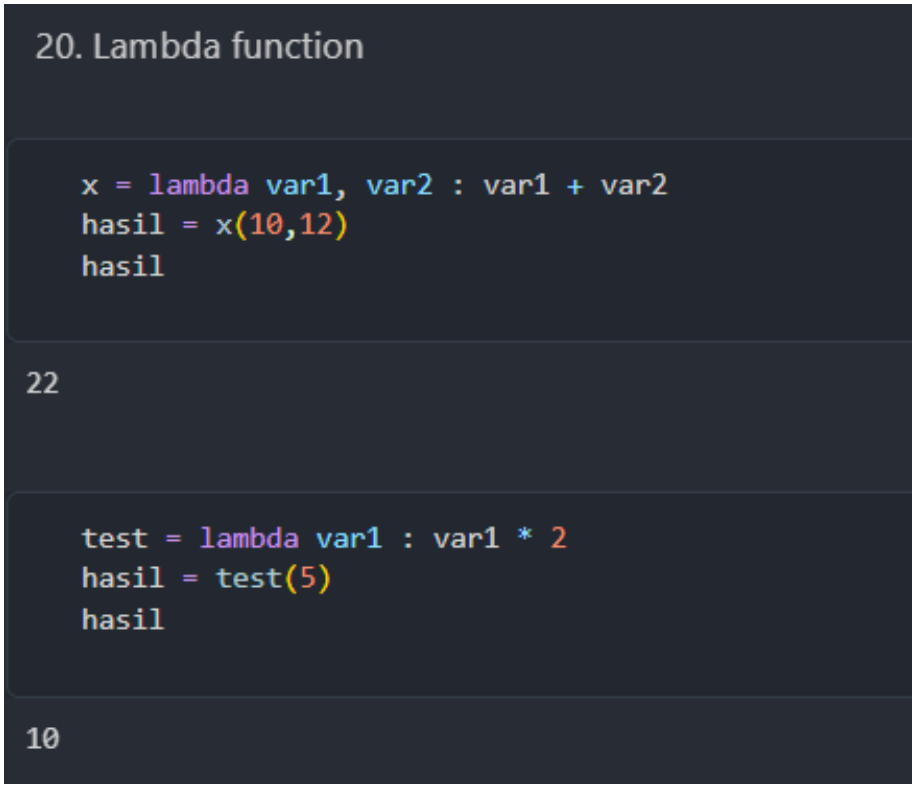
	Mhs_id	Nama	variable	value
0	001	Budi	Bhs Inggris	100
1	002	Joko	Bhs Inggris	90
2	003	Maya	Bhs Inggris	98
3	001	Budi	Matematika	90
4	002	Joko	Matematika	87
5	003	Maya	Matematika	72
6	001	Budi	Ekonomi	90
7	002	Joko	Ekonomi	80
8	003	Maya	Ekonomi	78

Gambar 62: Kode untuk melakukan melt pada DataFrame

Berdasarkan gambar 62, kolom yang ditentukan pada id_vars tetap dipertahankan, sedangkan kolom pada value_vars diubah menjadi dua kolom baru, yaitu variable dan value sehingga tiap nilai mata pelajaran ditampilkan sebagai baris terpisah.

C.2.20 Lambda Function

Penggunaan fungsi lambda dengan 2 parameter dapat diterapkan pada DataFrame untuk melakukan operasi tertentu. Gambar 63 adalah contoh kode untuk menggunakan fungsi lambda dengan 2 parameter pada DataFrame untuk melakukan operasi penjumlahan dan perkalian.



Gambar 63: Kode untuk menggunakan fungsi lambda dengan 2 parameter pada DataFrame

Berikut penggunaan fungsi lambda dengan kondisi bertingkat untuk menentukan nilai berdasarkan skor Matematika pada DataFrame.

```
mahasiswa = pd.DataFrame({
'Mhs_id':['001','002','003'],
'Nama': ['Budi', 'Joko', 'Maya'],
'Matematika': [90, 47, 72]})
mahasiswa
```

	Mhs_id	Nama	Matematika
0	001	Budi	90
1	002	Joko	47
2	003	Maya	72

```
mahasiswa['Grade'] = mahasiswa['Matematika'].apply(lambda nilai :
'A' if nilai >=90 else('B' if nilai >=70 and nilai <90 else ('C')))
mahasiswa
```

	Mhs_id	Nama	Matematika	Grade
0	001	Budi	90	A
1	002	Joko	47	C
2	003	Maya	72	B

Gambar 64: Kode untuk menggunakan fungsi lambda dengan kondisi bertingkat pada DataFrame

C.2.21 Membaca file

Untuk membaca file txt dapat menggunakan fungsi `pd.read_csv()` dengan memasukkan nama file sebagai argumen. Gambar 65 adalah contoh kode untuk membaca file txt.

21. Membaca file

```
import pandas as pd

df = pd.read_csv("data/datatest.txt")
df
```

	date	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
140	2015-02-02 14:19:00	23.700000	26.272000	585.200000	749.200000	0.004764	1
141	2015-02-02 14:19:59	23.718000	26.290000	578.400000	760.400000	0.004773	1
142	2015-02-02 14:21:00	23.730000	26.230000	572.666667	769.666667	0.004765	1
143	2015-02-02 14:22:00	23.722500	26.125000	493.750000	774.750000	0.004744	1
144	2015-02-02 14:23:00	23.754000	26.200000	488.600000	779.000000	0.004767	1
...	...	...	...	...	...	...	...
2800	2015-02-04 10:38:59	24.290000	25.700000	808.000000	1150.250000	0.004829	1
2801	2015-02-04 10:40:00	24.330000	25.736000	809.800000	1129.200000	0.004848	1
2802	2015-02-04 10:40:59	24.330000	25.700000	817.000000	1125.800000	0.004841	1
2803	2015-02-04 10:41:59	24.356667	25.700000	813.000000	1123.000000	0.004849	1
2804	2015-02-04 10:43:00	24.408333	25.681667	798.000000	1124.000000	0.004860	1

2665 rows × 7 columns

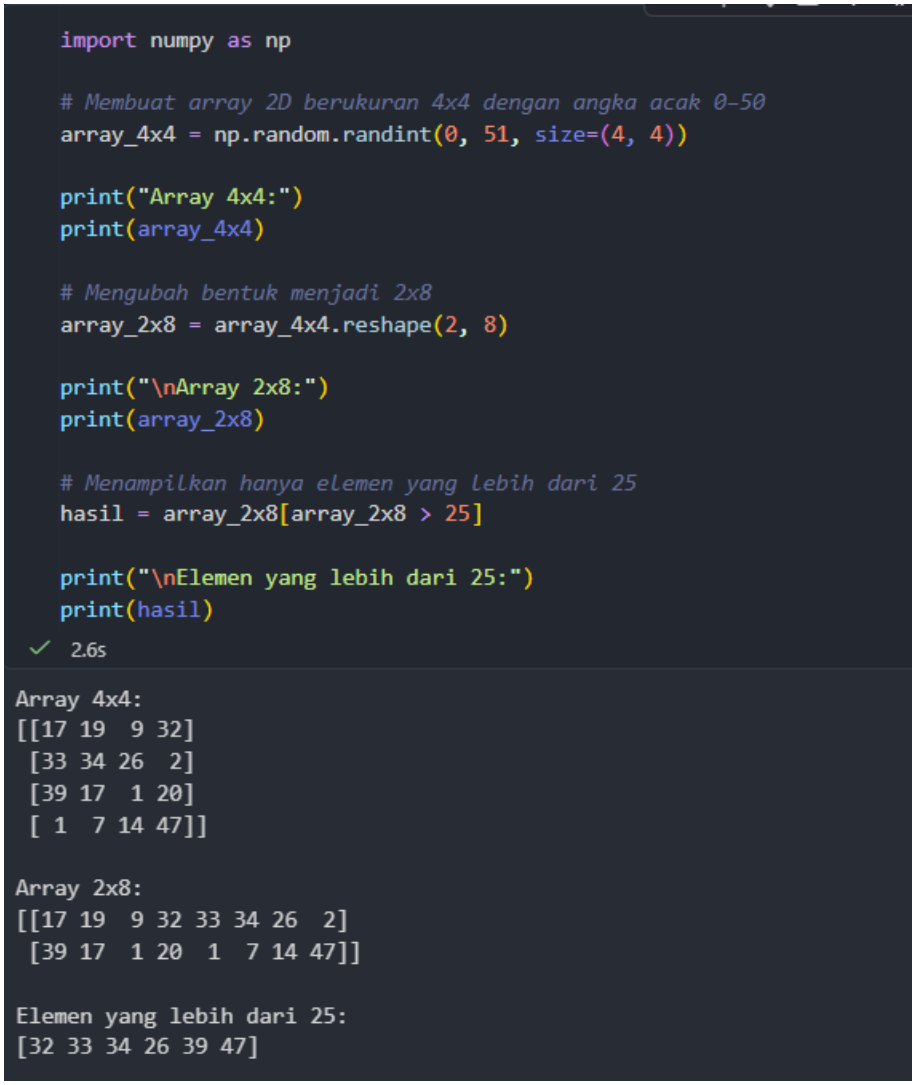
Gambar 65: Kode untuk membaca file txt

Berdasarkan gambar 65, praktikan mengunduh file dataset dari kaggle dengan link `https://www.kaggle.com/datasets/robmarkcole/occupancy-detection-data-set-uci?resource=download` lalu disimpan di dalam folder data. Kemudian, dataset dibaca dengan fungsi `pd.read_csv()`.

C.3 Tugas dan Analisis

C.3.1 Tugas 1

Buatlah array NumPy 2D berukuran 4×4 berisi angka acak (0–50), kemudian ubah bentuknya menjadi array 2×8 dan tampilkan hanya elemen-elemen yang bernilai lebih dari 25. Berikut penjelasannya:



Gambar 66: Ubah bentuk array NumPy 2D berukuran 4×4 menjadi 2×8

Array NumPy berukuran 4×4 dibuat menggunakan fungsi `np.random.randint(0, 51, size=(4` yang menghasilkan bilangan acak dari 0 hingga 50. Selanjutnya, fungsi `reshape(2,8)` digunakan untuk mengubah bentuk array menjadi 2×8 tanpa mengubah jumlah elemennya. Terakhir, operasi penyaringan `array_2x8[array_2x8 > 25]` digunakan untuk menampilkan hanya elemen yang memiliki nilai lebih dari 25.

C.3.2 Tugas 2

Buatlah dataframe untuk data karyawan yang berisi kolom Nama, Pekerjaan, dan Gaji. Lalu lakukan operasi groupby() pada kolom Pekerjaan dan hitung nilai

maksimum, minimum, serta standar deviasi untuk kolom ‘Gaji’ menggunakan agg(). Berikut penjelasannya:

```python
import pandas as pd

# Membuat DataFrame data karyawan
data = {
    'Nama': ['Andi', 'Budi', 'Citra', 'Dina', 'Eko', 'Fajar'],
    'Pekerjaan': ['Programmer', 'Programmer', 'Designer', 'Designer', 'Programmer', 'Designer'],
    'Gaji': [7000000, 8000000, 6500000, 7500000, 9000000, 7000000]
}

df = pd.DataFrame(data)

# Groupby berdasarkan Pekerjaan dan menghitung statistik Gaji
hasil = df.groupby('Pekerjaan')['Gaji'].agg(['max', 'min', 'std'])

print(df)
print("\nHasil groupby:")
print(hasil)
```

✓ 2.9s

```
    Nama   Pekerjaan     Gaji
0   Andi  Programmer  7000000
1   Budi  Programmer  8000000
2  Citra    Designer  6500000
3   Dina    Designer  7500000
4    Eko  Programmer  9000000
5  Fajar    Designer  7000000

Hasil groupby:
                max      min        std
Pekerjaan
Designer    7500000  6500000   500000.0
Programmer  9000000  7000000  1000000.0
```

Gambar 67: Operasi groupby() pada kolom Pekerjaan dan hitung nilai maksimum, minimum, serta standar deviasi untuk kolom ‘Gaji’ menggunakan agg()

Data karyawan disimpan dalam sebuah *DataFrame* yang memiliki tiga kolom, yaitu `Nama`, `Pekerjaan`, dan `Gaji`. Selanjutnya, fungsi `groupby()` digunakan untuk mengelompokkan data berdasarkan kolom `Pekerjaan`. Fungsi `agg()` kemudian digunakan pada kolom `Gaji` untuk menghitung nilai maksimum (`max`), minimum (`min`), dan standar deviasi (`std`) dari setiap kelompok pekerjaan. Dengan demikian, dapat diketahui statistik gaji untuk masing-masing jenis pekerjaan.

C.3.3 Tugas 3

Buat dua DataFrame untuk data karyawan dengan kolom Emp_id, Nama, dan Divisi, serta DataFrame gaji dengan kolom ID, Gaji, dan Status (masing-masing minimal tiga baris). Setelah itu, gabungkan kedua DataFrame tersebut menggunakan merge() dengan left_on='Emp_id' dan right_on='ID', terapkan left join, dan gunakan suffixes untuk membedakan kolom yang sama. Berikut penjelasannya:

```python
import pandas as pd

# DataFrame karyawan
karyawan = pd.DataFrame({
    'Emp_id': [101, 102, 103, 104],
    'Nama': ['Andi', 'Budi', 'Citra', 'Dina'],
    'Divisi': ['IT', 'HR', 'Finance', 'IT']
})

# DataFrame gaji
gaji = pd.DataFrame({
    'ID': [101, 102, 103],
    'Gaji': [7000000, 8000000, 7500000],
    'Status': ['Tetap', 'Kontrak', 'Tetap']
})

# Menggabungkan kedua DataFrame dengan left join
hasil = pd.merge(
    karyawan,
    gaji,
    left_on='Emp_id',
    right_on='ID',
    how='left',
    suffixes=('_karyawan', '_gaji')
)

print(hasil)
```

✓ 0.0s

	Emp_id	Nama	Divisi	ID	Gaji	Status
0	101	Andi	IT	101.0	7000000.0	Tetap
1	102	Budi	HR	102.0	8000000.0	Kontrak
2	103	Citra	Finance	103.0	7500000.0	Tetap
3	104	Dina	IT	NaN	NaN	NaN

Gambar 68: Gabungkan dua DataFrame menggunakan merge() dengan left join dan suffixes untuk membedakan kolom yang sama

Dua *DataFrame* dibuat untuk menyimpan data karyawan dan data gaji. DataFrame

karyawan memiliki kolom `Emp_id`, `Nama`, dan `Divisi`, sedangkan DataFrame gaji memiliki kolom `ID`, `Gaji`, dan `Status`. Kedua DataFrame kemudian digabungkan menggunakan fungsi `merge()` dengan `left_on='Emp_id'` dan `right_on='ID'`. Parameter `how='left'` digunakan untuk mempertahankan seluruh data dari DataFrame karyawan, sedangkan data gaji yang tidak memiliki pasangan akan memiliki nilai `NaN`. Parameter `suffixes` digunakan untuk membedakan nama kolom yang sama apabila terdapat kolom dengan nama identik pada kedua DataFrame.

D Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa NumPy dan Pandas adalah library Python yang sangat berguna untuk analisis data. NumPy menyediakan struktur data array yang efisien dan berbagai fungsi untuk manipulasi array, seperti slicing, reshaping, dan operasi matematika. Sementara itu, Pandas menawarkan struktur data DataFrame yang memudahkan pengolahan data tabular, termasuk operasi filtering, grouping, merging, dan pivoting. Dengan memahami kedua library ini, praktikan dapat melakukan analisis data dengan lebih cepat dan efektif.

Berdasarkan tugas yang telah dikerjakan, praktikan dapat membuat array NumPy 2D, mengubah bentuknya, dan melakukan filtering elemen berdasarkan kondisi tertentu. Selain itu, praktikan juga dapat membuat DataFrame untuk data karyawan, melakukan operasi groupby untuk menghitung statistik gaji, serta menggabungkan dua DataFrame menggunakan merge dengan berbagai parameter. Praktikum ini memberikan pemahaman yang lebih baik tentang manipulasi data menggunakan NumPy dan Pandas, yang merupakan keterampilan penting dalam analisis data.

E Daftar Pustaka

[1] D. Intern, "Python: Pengertian, Contoh Penggunaan, dan Manfaat Mempelajarinya — dicoding.com," https://www.dicoding.com/blog/python-pengertian-contoh-penggunaan-dan-manfaat-mempelajarinya/, 2023, [Accessed 21-08-2026].

[2] Prasatya, "Apa Itu NumPy dalam Python? Pengertian dan Contoh Kode untuk Pemula - CODEPOLITAN — codepolitan.com," https://www.codepolitan.com/blog/apa-itu-numpy-dalam-python-pengertian-dan-contoh-kode-untuk-pemula/, 2025, [Accessed 21-08-2026].

[3] R. T. Team, "Apa Itu Pandas? Definisi & Penjelasan — REA Ruangguru — ruangguru.com," https://www.ruangguru.com/rea/kamus/p/pandas, [Accessed 21-08-2026].

[4] "Panduan Lengkap Pandas Dataframe — digitalskola.com," https://digitalskola.com/blog/home/pandas-dataframe, 2024, [Accessed 21-08-2026].